Binary Search Trees: Basic Operations

Find

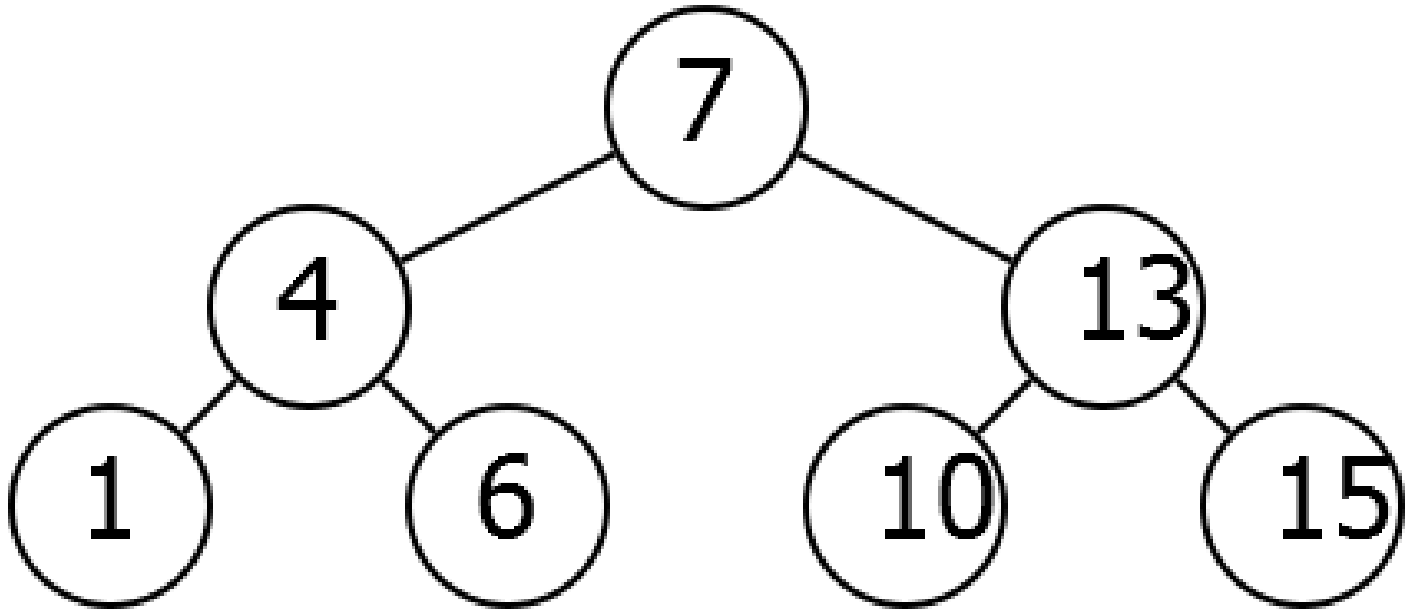
Find

Input: Key k, Root R

Output: The node in the tree of R with key k

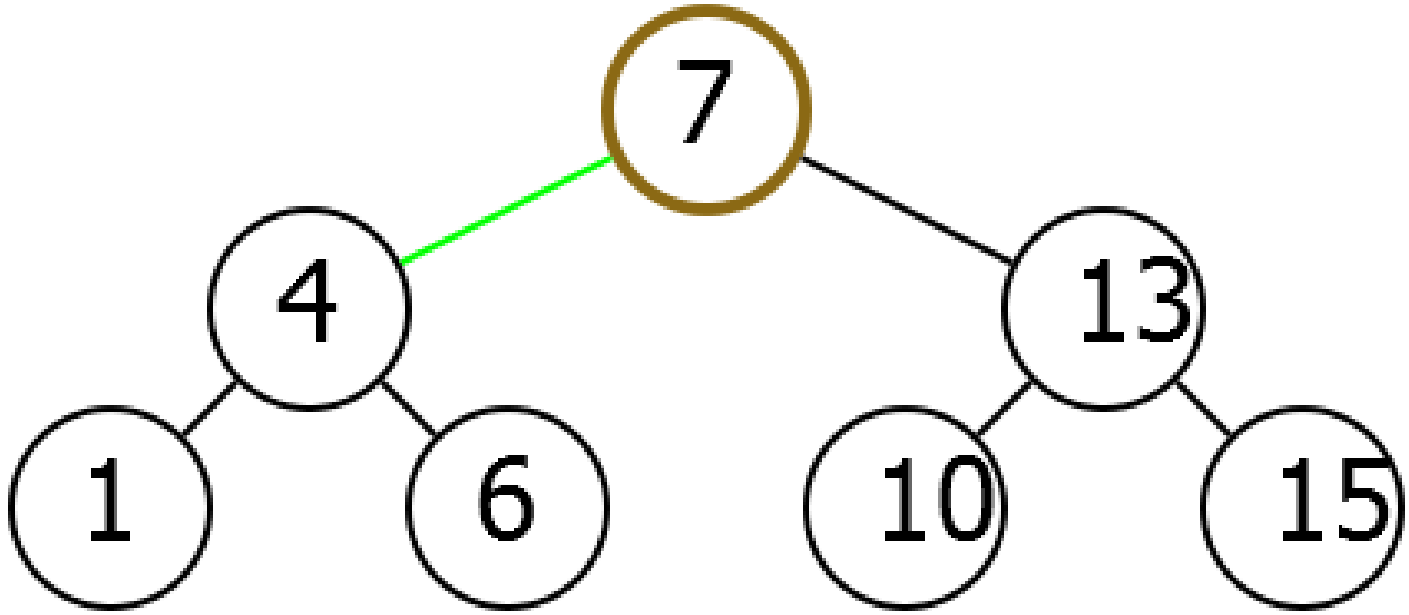
Idea

`Find`(6)



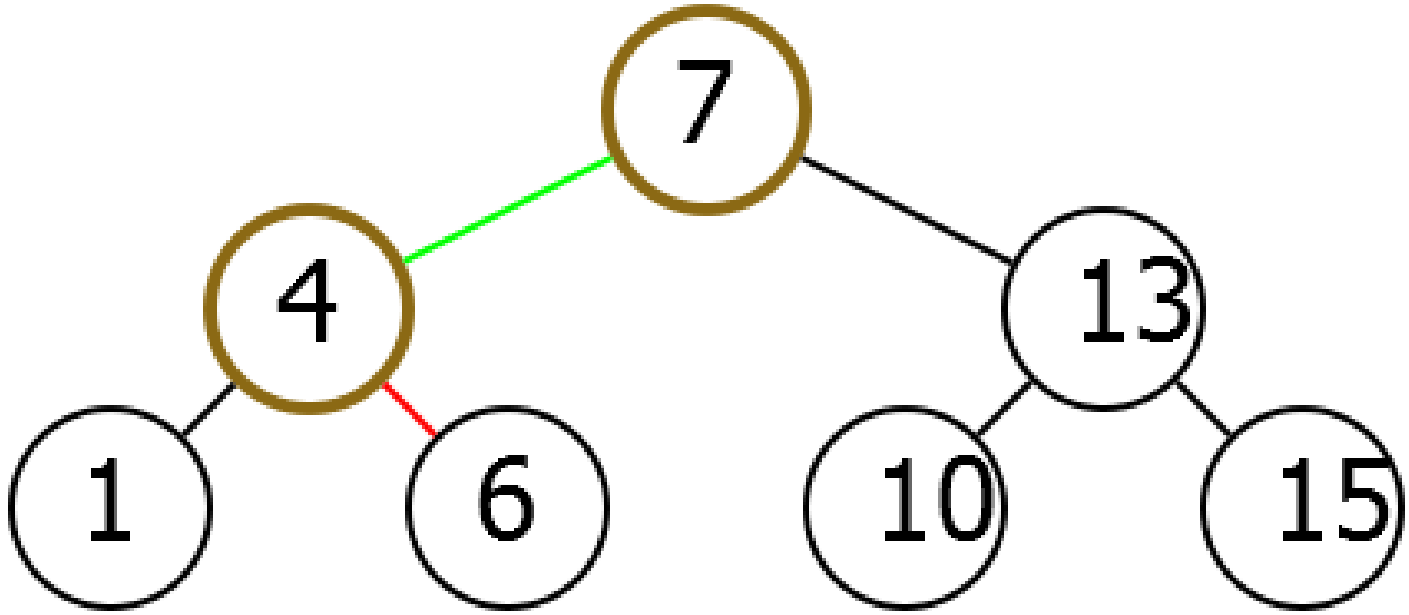
Idea

`Find`(6)



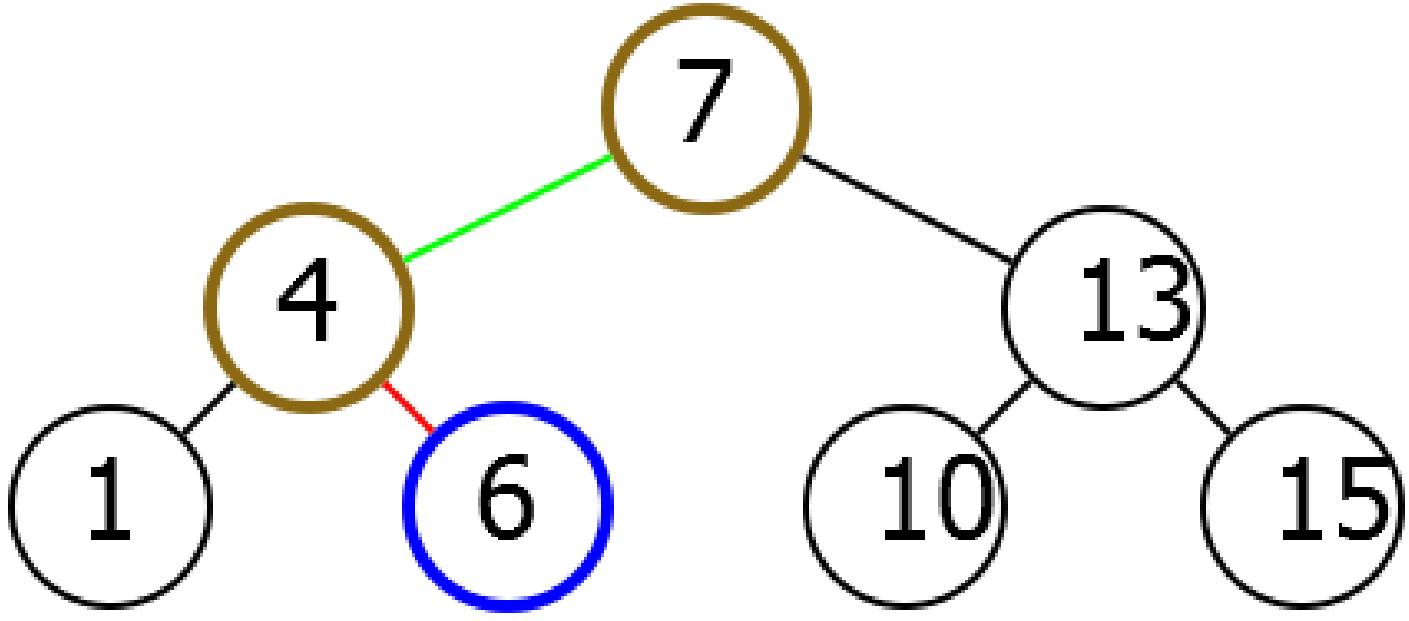
Idea

`Find`(6)



Idea

`Find`(6)



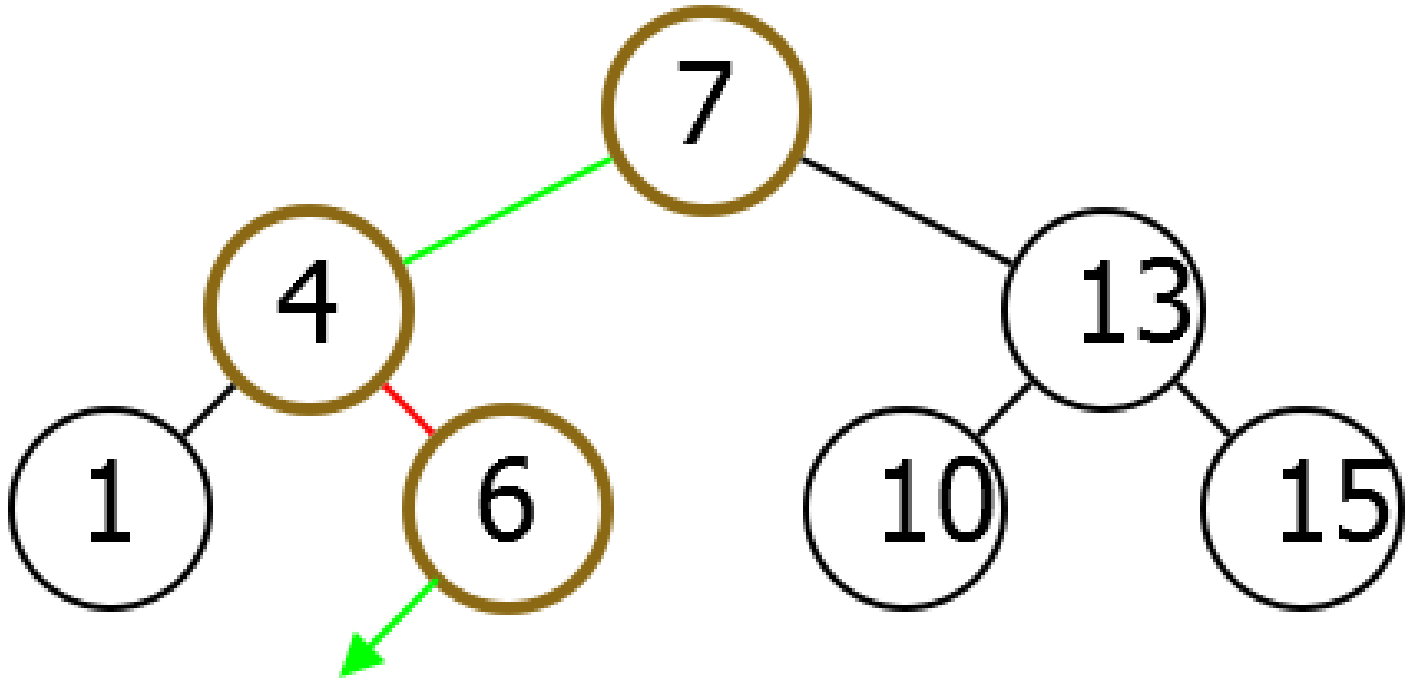
Algorithm

Find(k, R)

```
if R.Key = k:
    return R
else if R.Key > k :
    return Find(k, R.Left)
else if R.Key < k :
    return Find(k, R.Right)
```

Missing Key

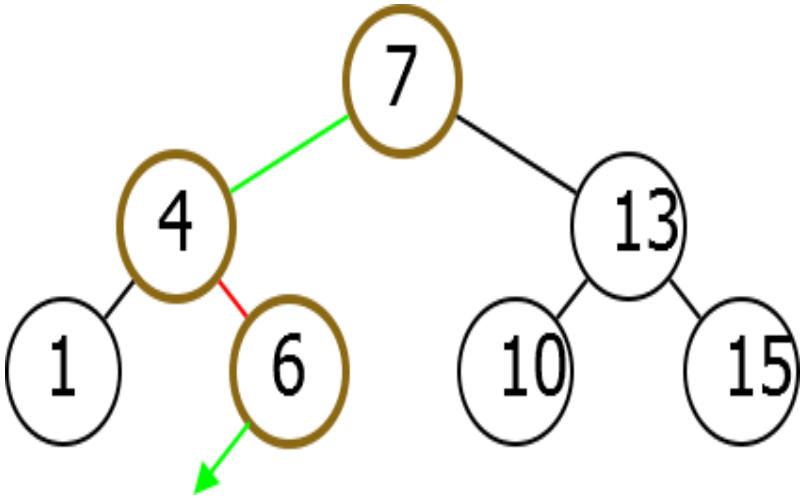
Run `Find`(5).



Key not in tree. Did find point where it should be.

Missing Key

Run `Find`(5).



If you search for k= 5, the flow would be:

1.Compare k= 5 with R.Key= 7: 5<7, move to the left subtree.

2.Compare k= 5 with R.Key=4: 5>4, move to the right subtree.

3.Compare k=5 with R.Key=6: 5<6, move to the left subtree.

4. R becomes null. Return "Key not found.

Key not in tree. Did find point where it should be.

Modified Algorithm

```
Function Find(k, R):


if R is null:
        return "Key not found"  # Handle case when tree is empty or key is not found

    if R.Key == k:
        return R  # Key found

    else if R.Key > k:
        return Find(k, R.Left)  # Search in the left subtree

    else:
        return Find(k, R.Right)  # Search in the right subtree
```

Insert

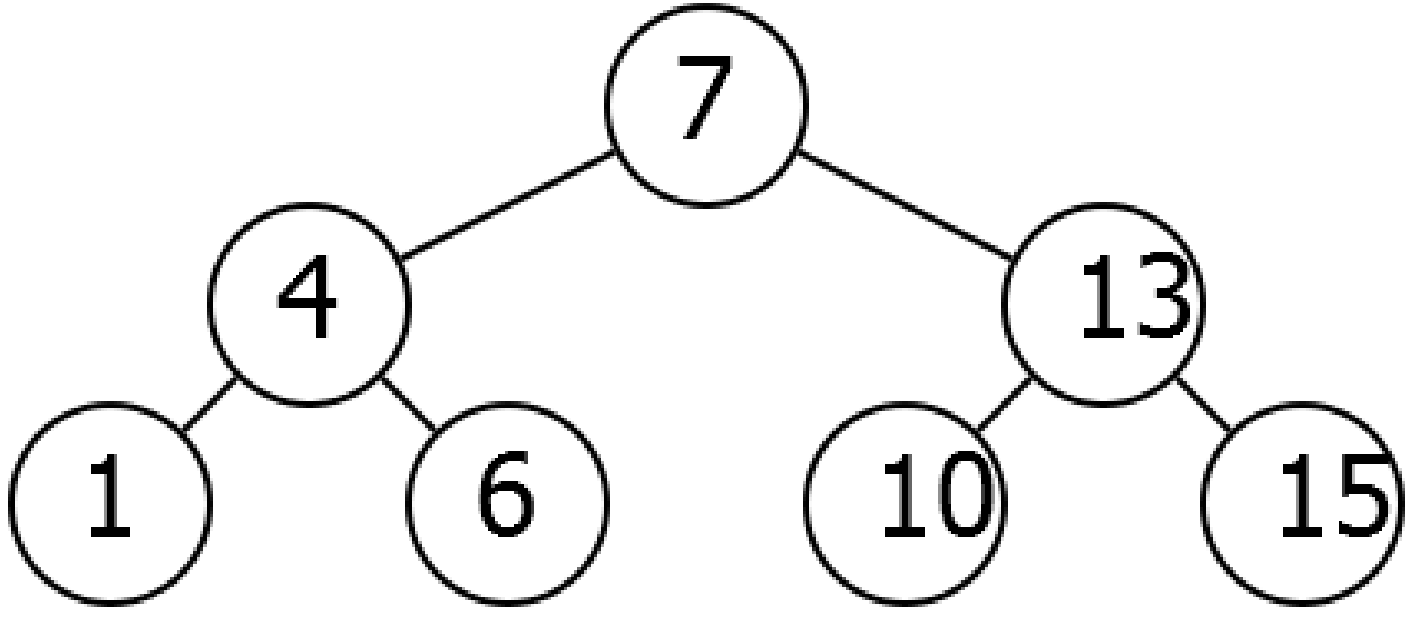
Insert

Input: Key k and root R

Output: Adds node with key k to the tree

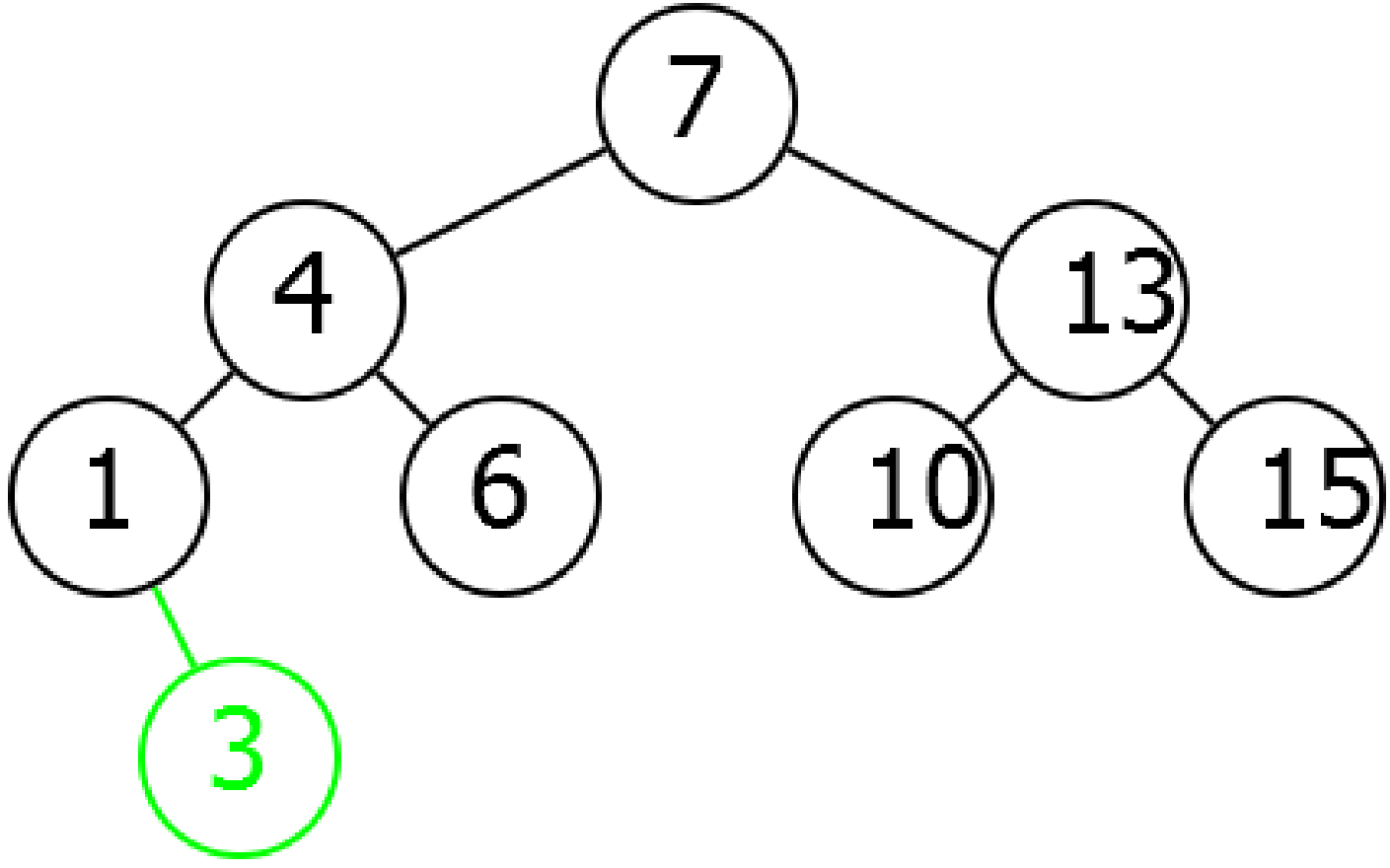
Insert Idea

`Insert`(3)



Insert Idea

`Insert`(3)



Implementation

Insert(k, R)

$P \leftarrow$ `Find`(k, R)

`Add new node with key` k `as child of` P

Delete

Delete

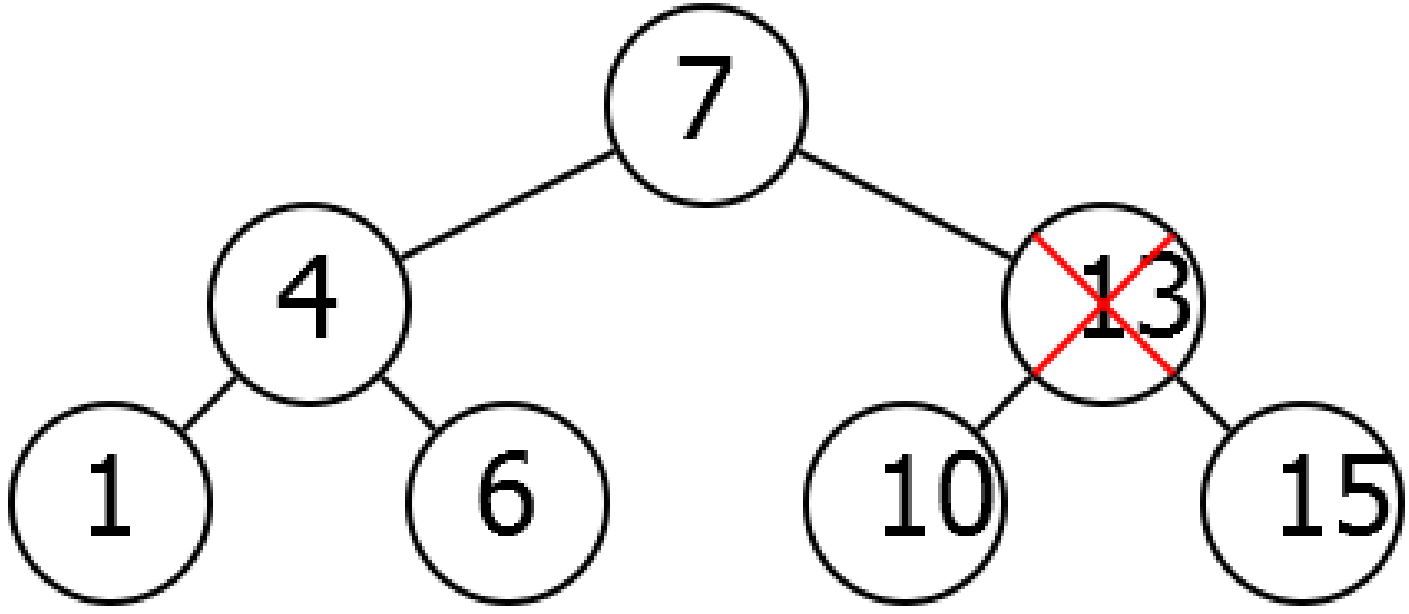
Input: Node *N*

Output: Removes node *N* from the tree

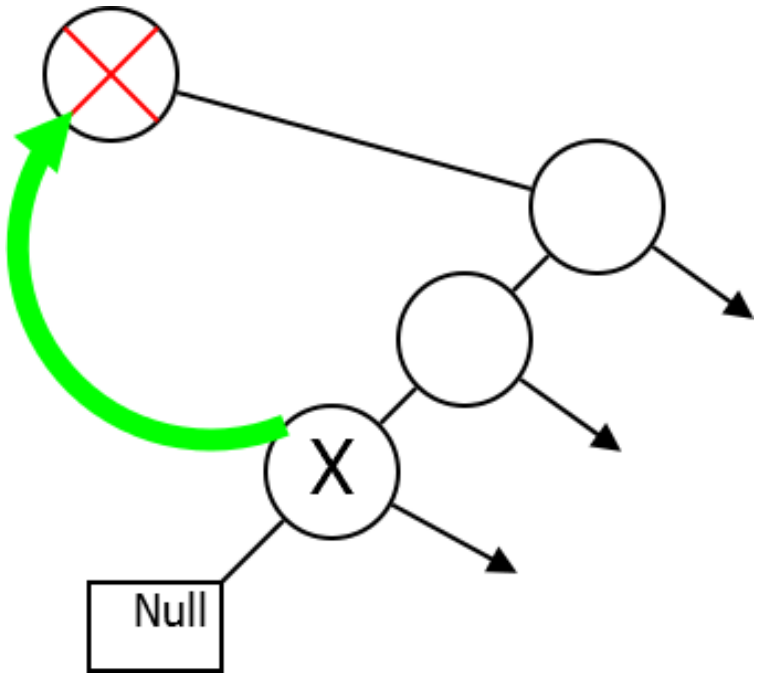
Difficulty

Cannot simply remove.

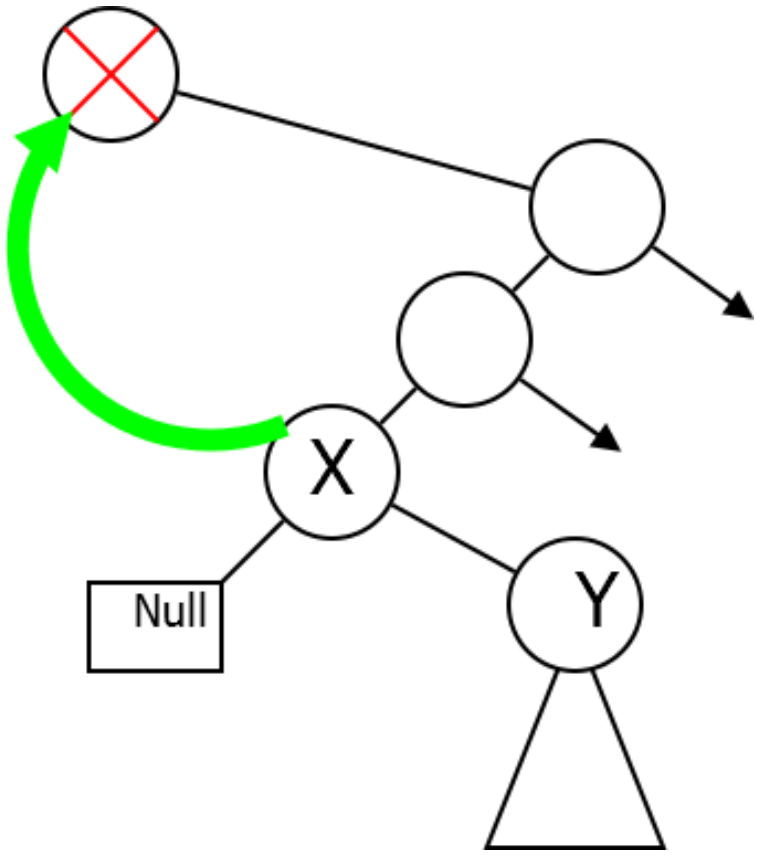
`Delete`(13)



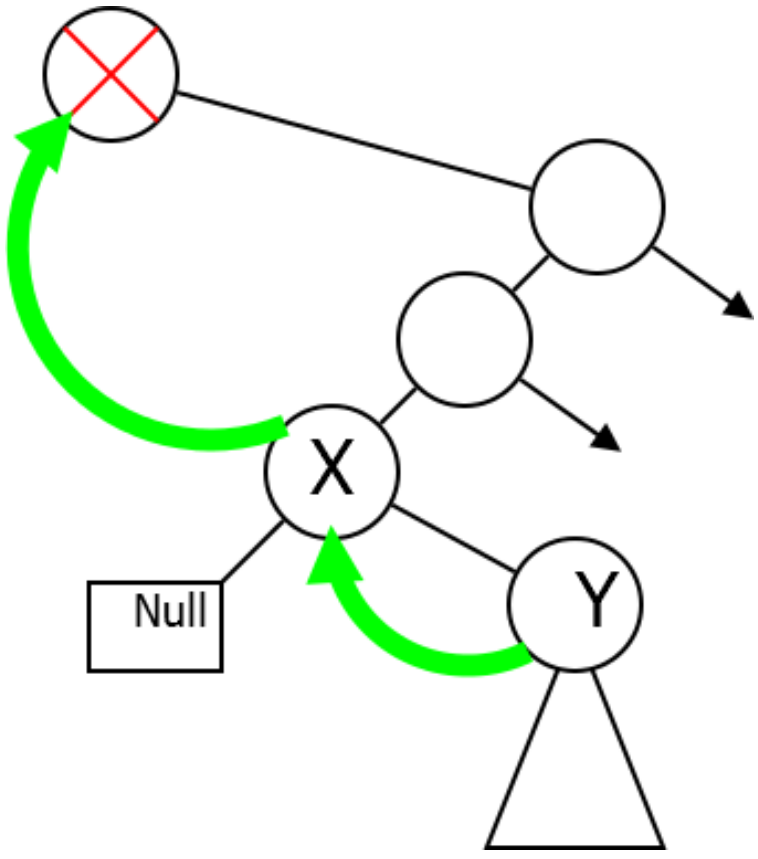
Idea



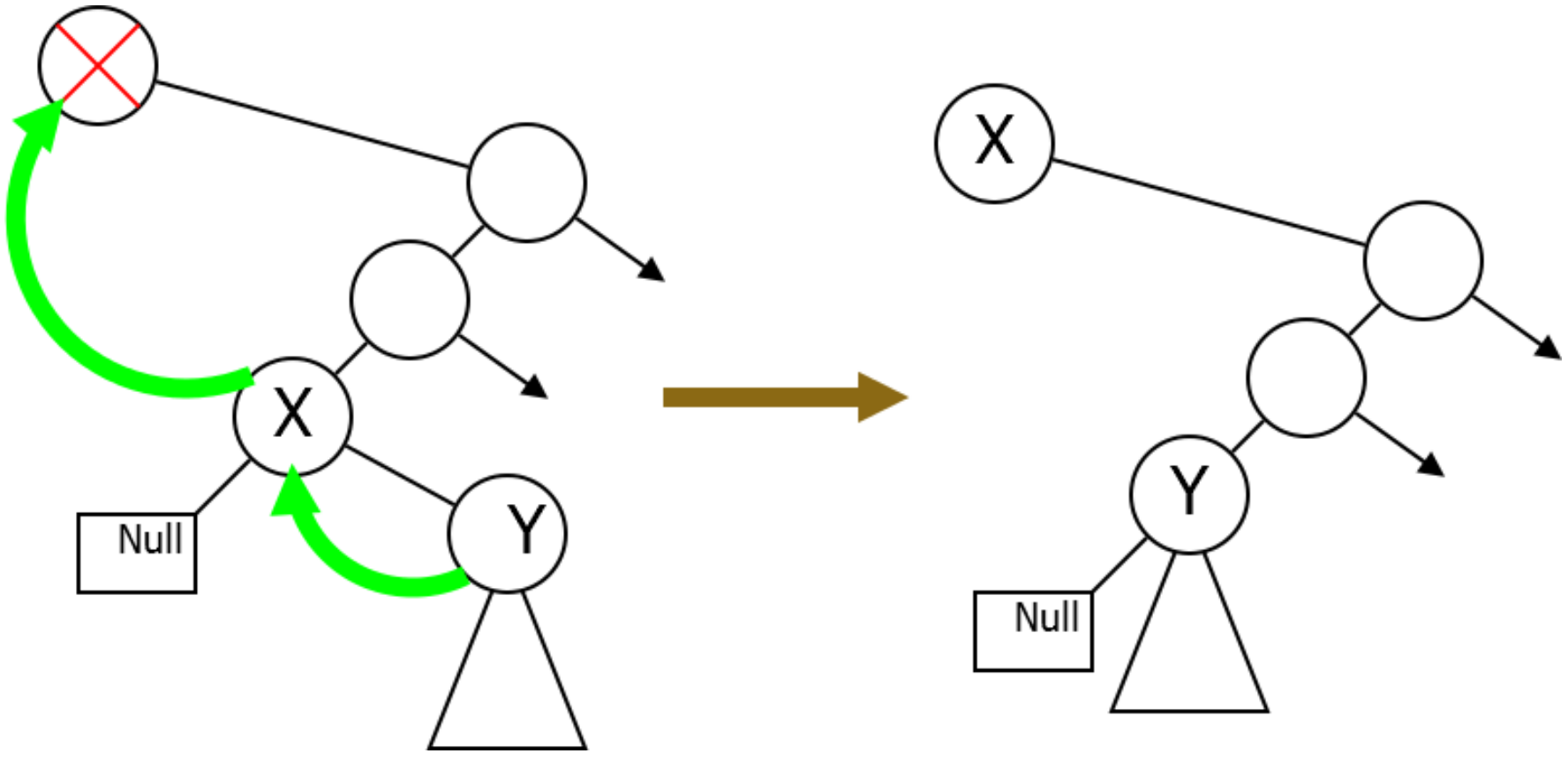
Idea



Idea

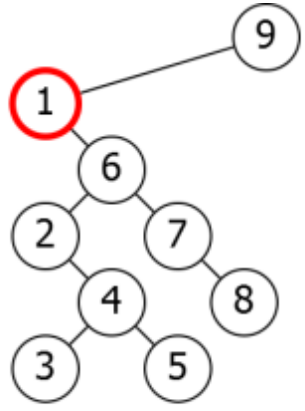


Idea

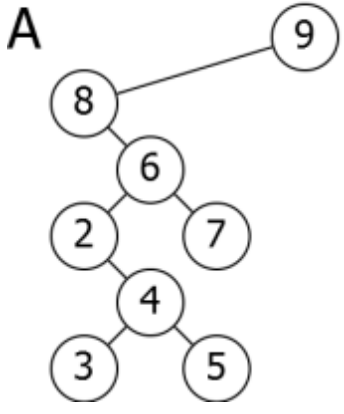


Problem

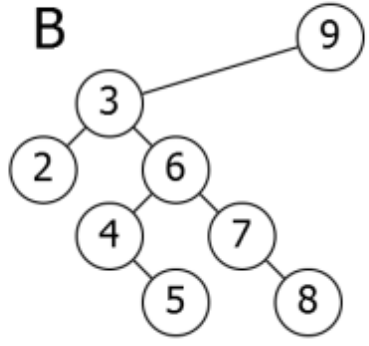
Which of the following trees is obtained when the selected node is deleted?



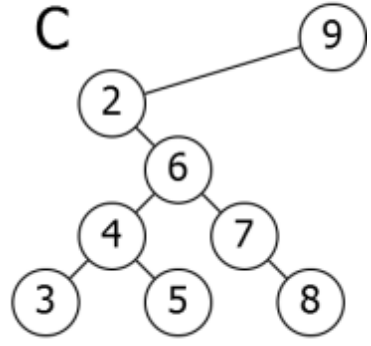
A



B

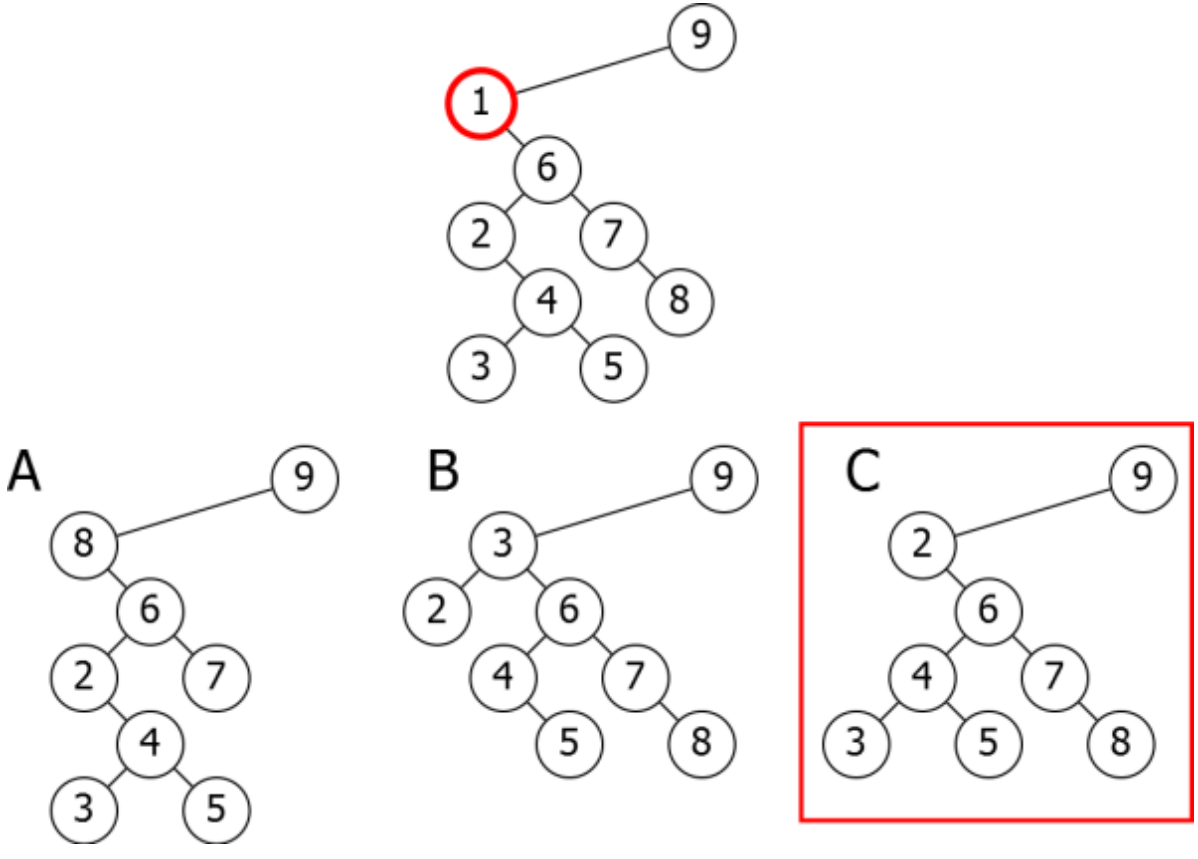


C



Problem

Which of the following trees is obtained when the selected node is deleted?



AVL Tree

- An AVL tree is a type of self-balancing binary search tree named after its inventors Adelson-Velsky and Landis.

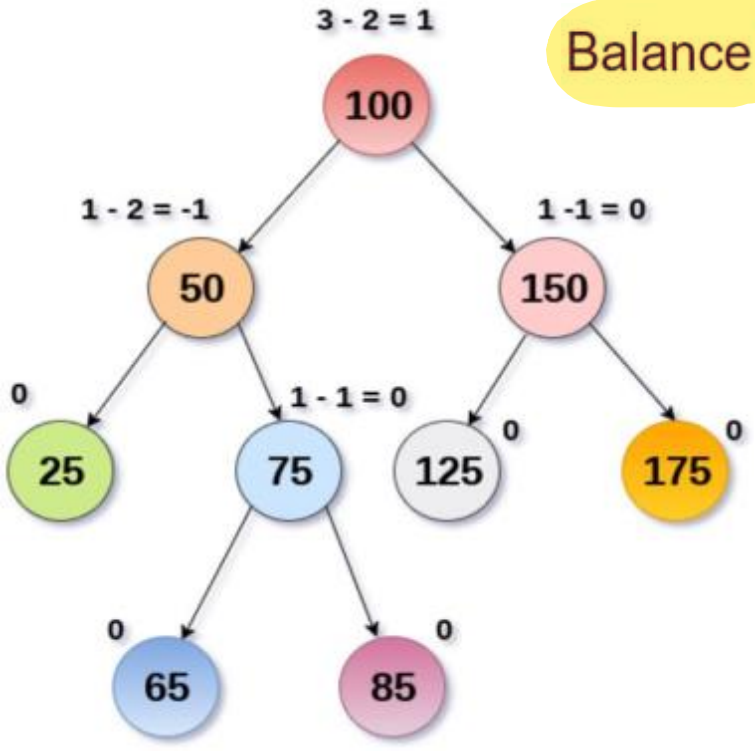
- It is designed to maintain a balanced tree structure by automatically reorganizing itself as nodes are added or removed from the tree.
  - The AVL tree is balanced by ensuring that the height difference between the left and right subtrees of any node in the tree is no more than one.

  - This ensures that the worst-case time complexity for common operations such as searching, insertion, and deletion is O(log n).

- It is used in many applications, including database indexing, compiler design, and computer graphics.

AVL Tree

The AVL tree is balanced by ensuring that the height difference between the left and right subtrees of any node in the tree is no more than one.

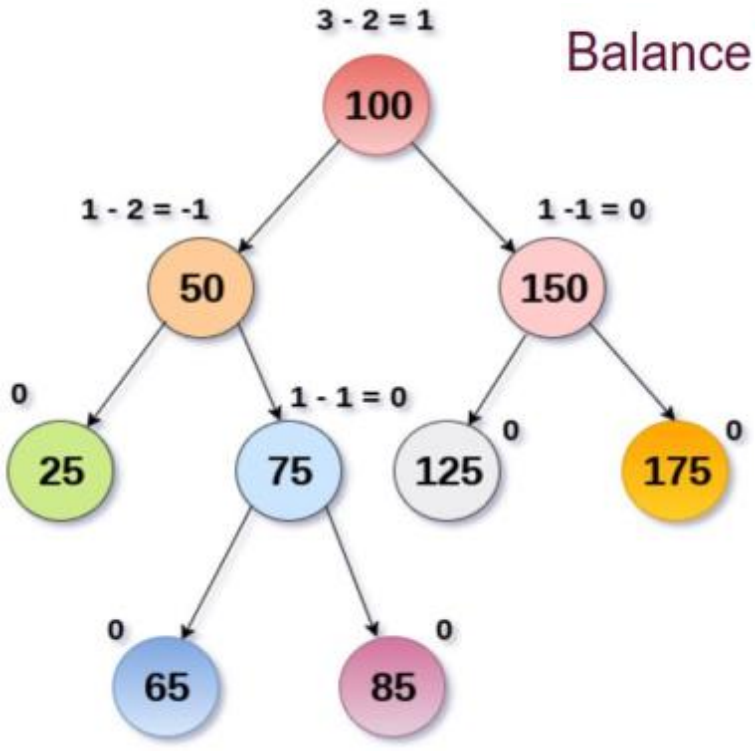


Balance Factor (k) = height (left(k)) - height (right(k))

- If balance factor of any node is 1:
  - Then the left sub-tree is one level higher than the right sub-tree.
- If balance factor of any node is 0
  - Then the left sub-tree and right sub-tree contain equal height.
- If balance factor of any node is -1,
  - it means that the left sub-tree is one level lower than the right sub-tree.

AVL Tree (cont.)

The AVL tree is balanced by ensuring that the height difference between the left and right subtrees of any node in the tree is no more than one.



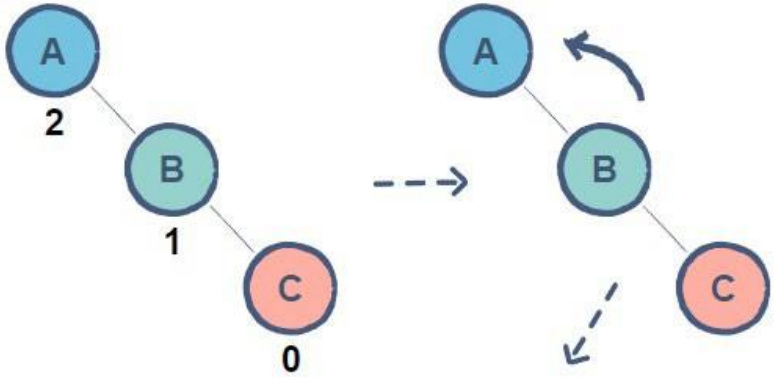
Balance Factor (k) = height (left(k)) - height (right(k))

- AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree.
- Searching and traversing do not lead to the violation in property of AVL tree.
- Insertion and deletion are the operations which can violate this property and therefore, they need to be revisited

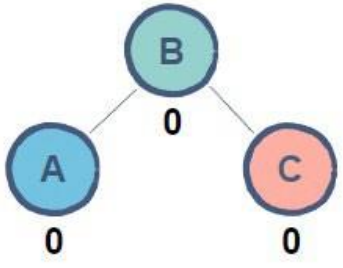
AVL Rotations

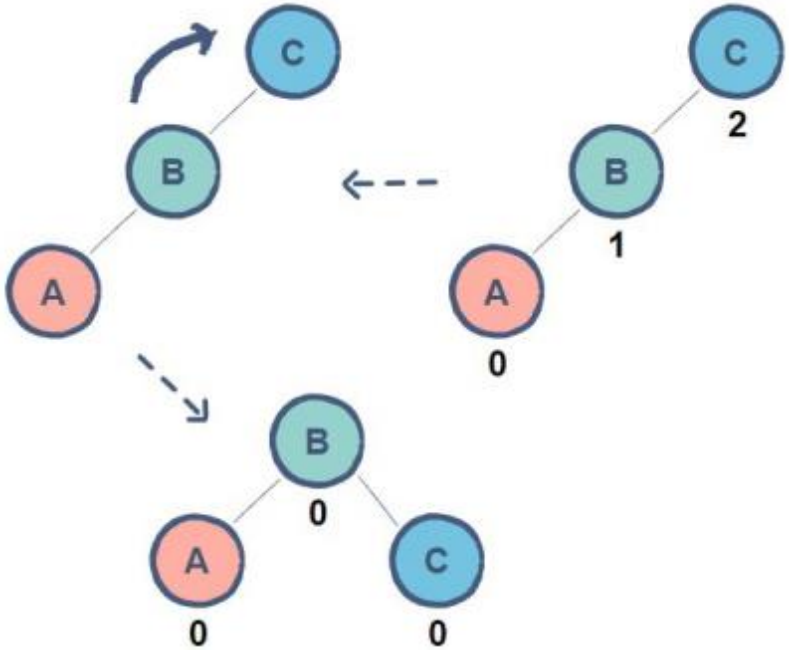
- Left rotation
- Right rotation
- Left-Right rotation
- Right-Left rotation

AVL Rotations



Left rotation



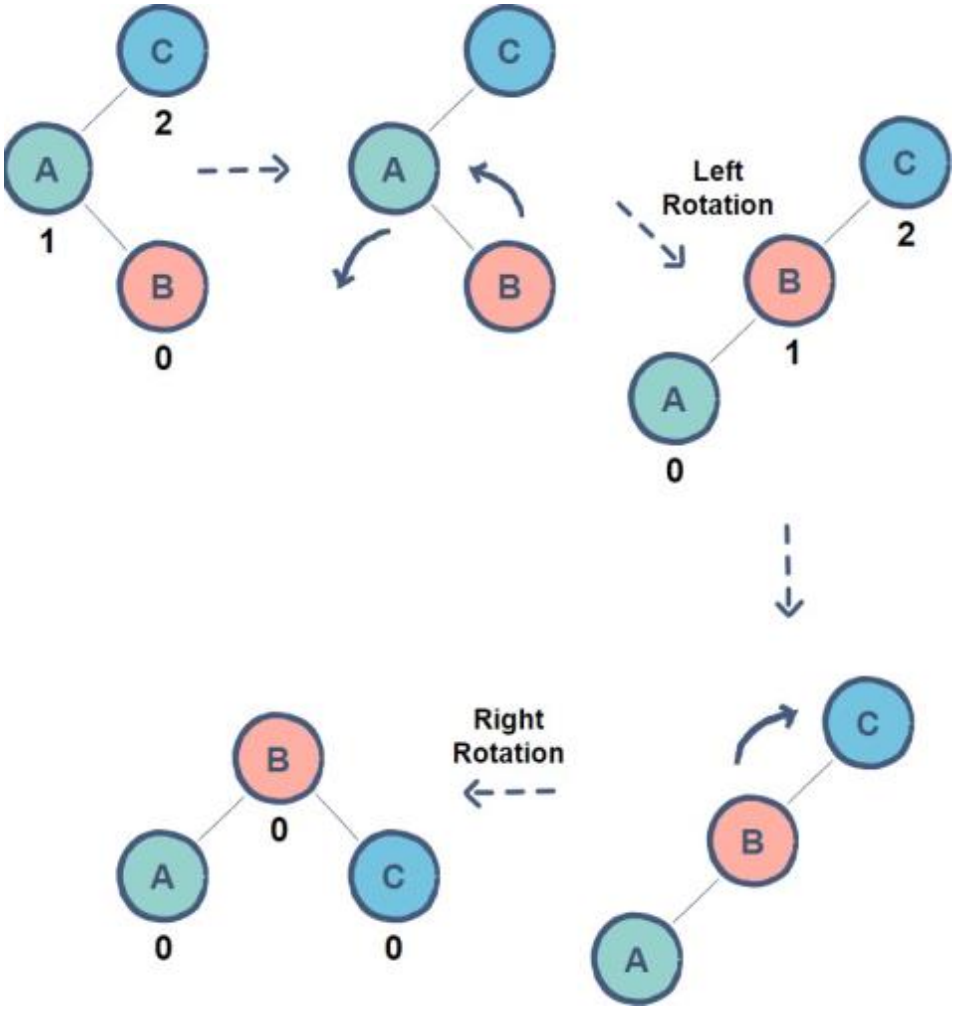


Right rotation

- A single rotation applied when a node is inserted in the right subtree of a right subtree.
- **node A** has a balance factor of 2 after the insertion of **node C**.
- By rotating the tree left, **node B** becomes the root resulting in a balanced tree.

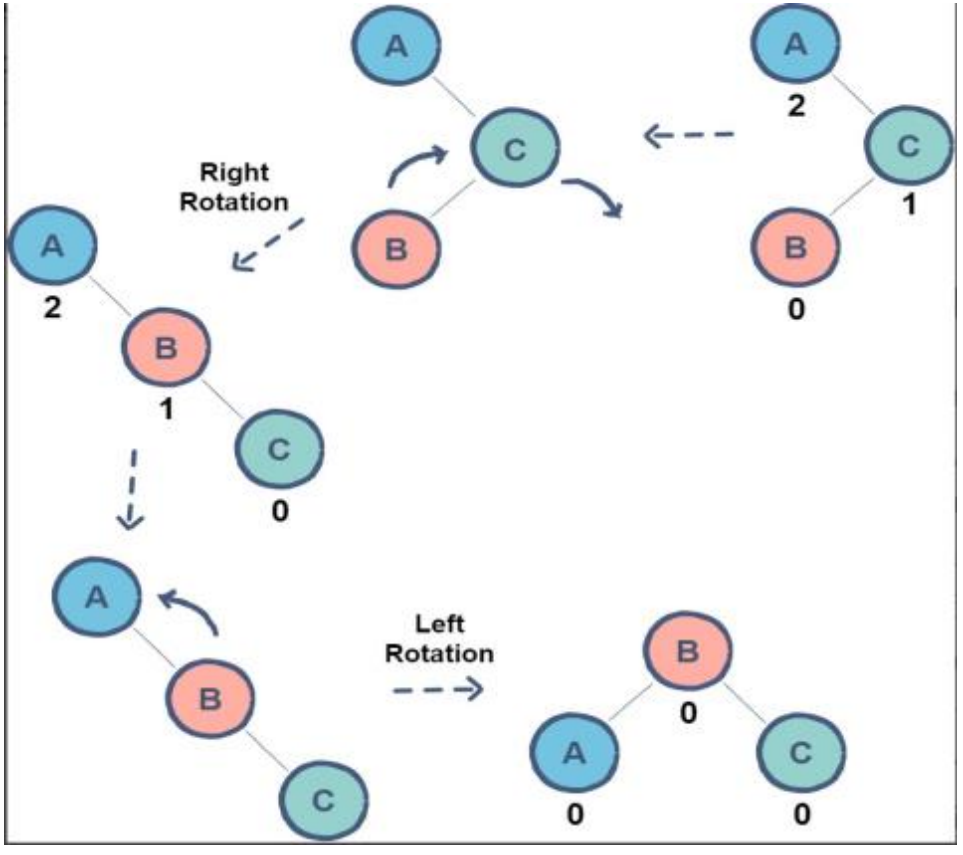
Left-Right Rotation

- A double rotation in which a *left rotation* is followed by a *right rotation*.
- In the given example, **node B** is causing an imbalance resulting in **node C** to have a balance factor of 2.
- As **node B** is inserted in the right subtree of **node A**, a *left rotation* needs to be applied.
- However, a single rotation will not give us the required results. Now, all we have to do is apply the *right rotation* as shown before to achieve a balanced tree.



Right-Left Rotation

- A double rotation in which a *right rotation* is followed by a *left rotation*.
- In the given example, **node B** is causing an imbalance resulting in **node A** to have a balance factor of 2.
- As **node B** is inserted in the left subtree of **node C**, a *right rotation* needs to be applied.
- a single rotation will not give us the required results.
- Now, by applying the *left rotation* as shown before, we can achieve a balanced tree.



Left-Left Imbalance – Solution Right Rotation

- This occurs when the balance factor of a node is +2
- and its left child's balance factor is +1.
- To fix this imbalance, perform a right rotation on the unbalanced node.
- This moves the left child up to become the new root, and the unbalanced node becomes the right child of the new root.

Right-Right Imbalance – Solution Left Rotation

- This occurs when the balance factor of a node is -2
- and its right child's balance factor is -1.
- To fix this imbalance, perform a left rotation on the unbalanced node.
- This moves the right child up to become the new root, and the unbalanced node becomes the left child of the new root.

Left-Right Imbalance – Solution Left-Right Double Rotation

- This occurs when the balance factor of a node is +2
- and its left child's balance factor is -1.
- To fix this imbalance, perform a left-right double rotation.
- This involves performing a left rotation on the left child, followed by a right rotation on the unbalanced node.

Right-Left Imbalance – Solution Left Rotation

- occurs when the balance factor of a node is -2
- and its right child's balance factor is +1.
- To fix this imbalance, perform a right-left double rotation.
- This involves performing a right rotation on the right child, followed by a left rotation on the unbalanced node.

Extended Example

Insert 3,2,1,4,5,6,7, 16,15,14

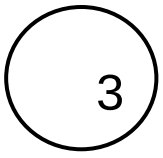


Fig 1

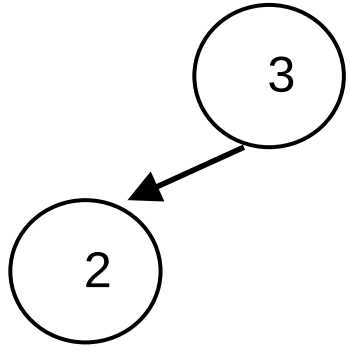


Fig 2

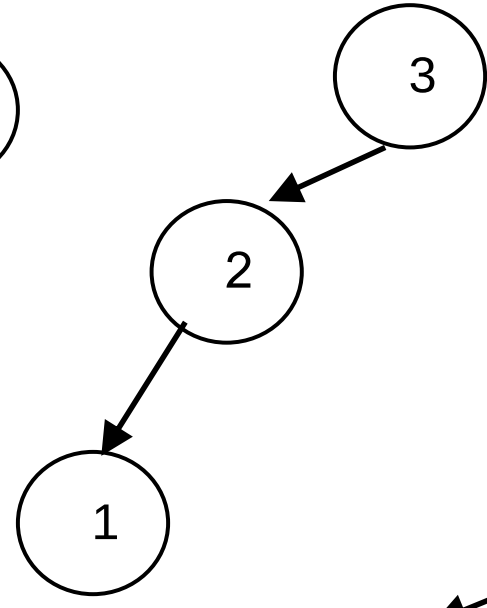


Fig 3

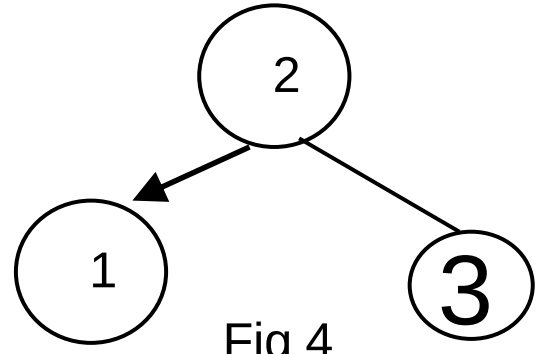


Fig 4

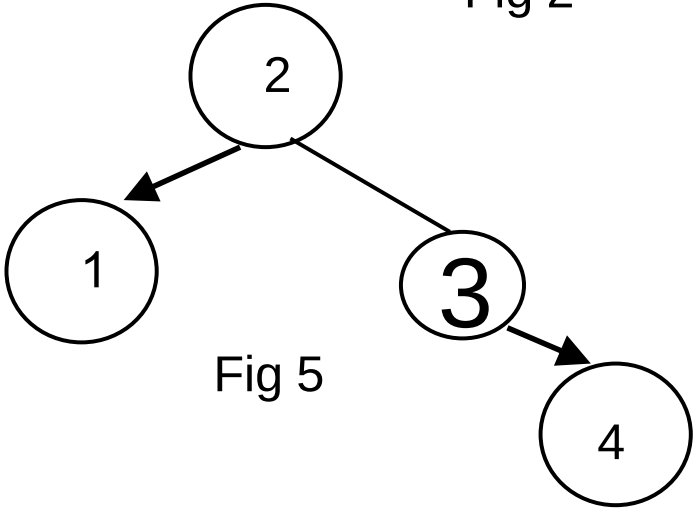


Fig 5

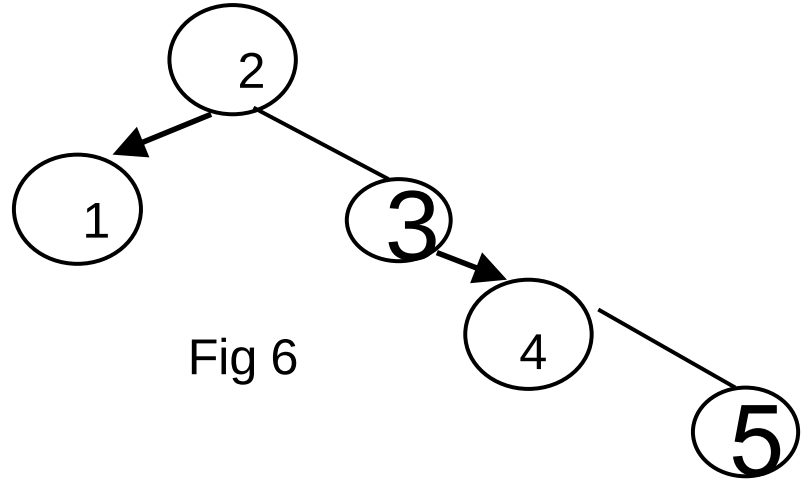


Fig 6

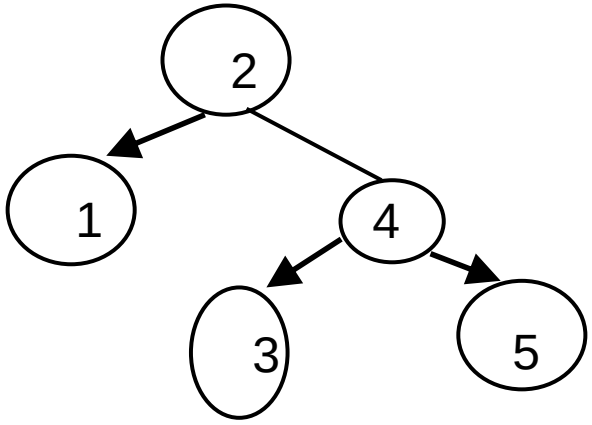


Fig 7

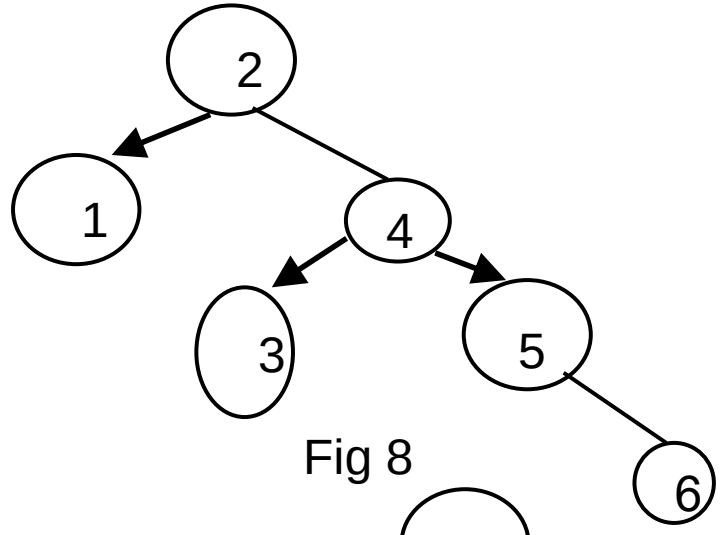


Fig 8

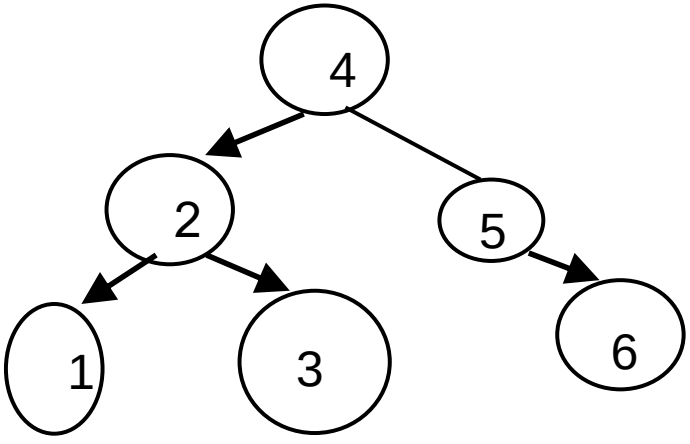


Fig 9

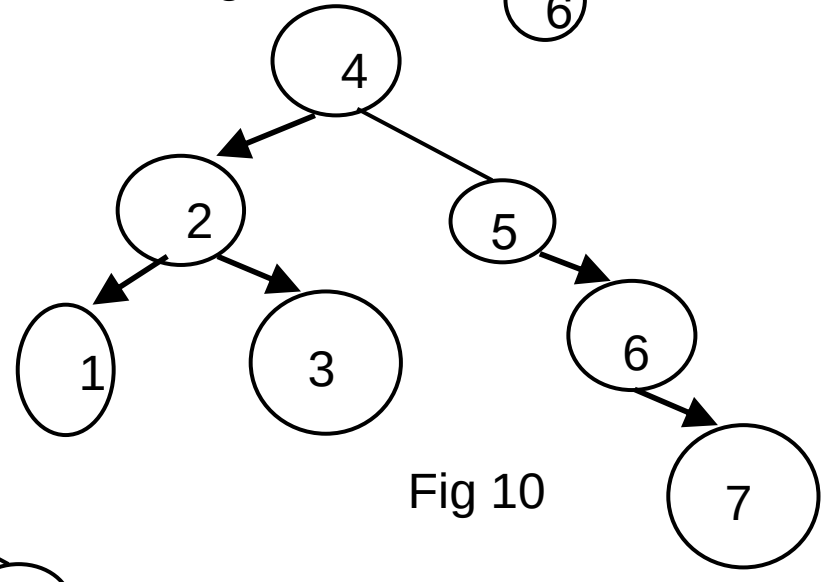


Fig 10

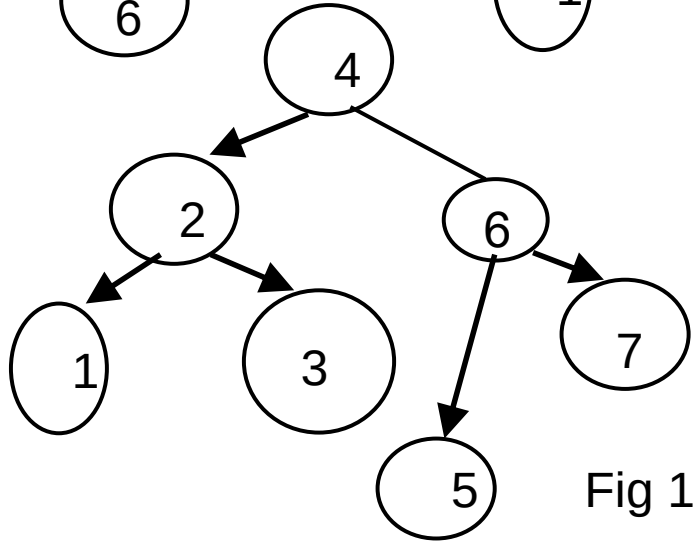


Fig 11

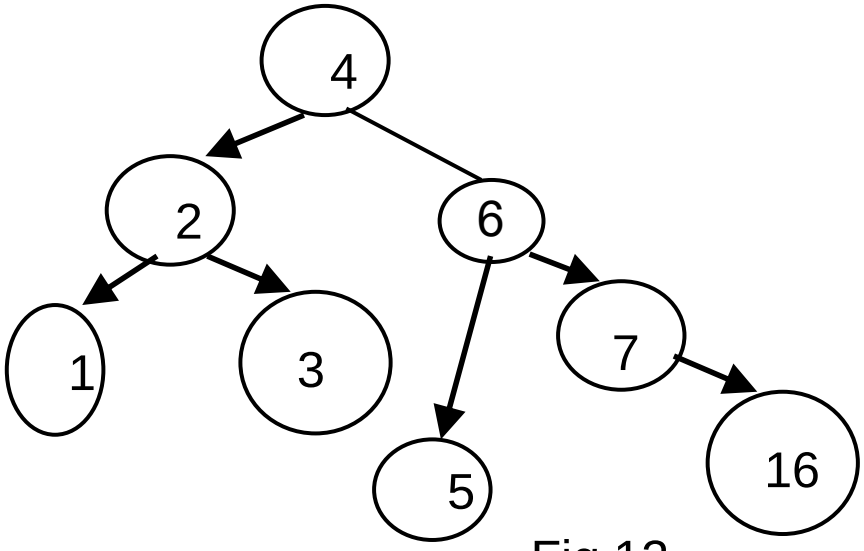


Fig 12

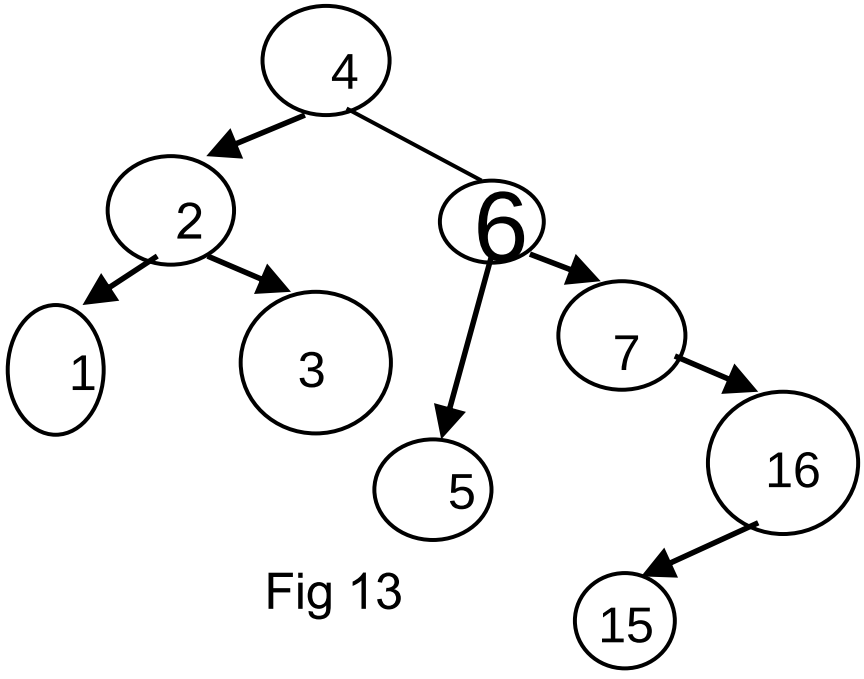


Fig 13

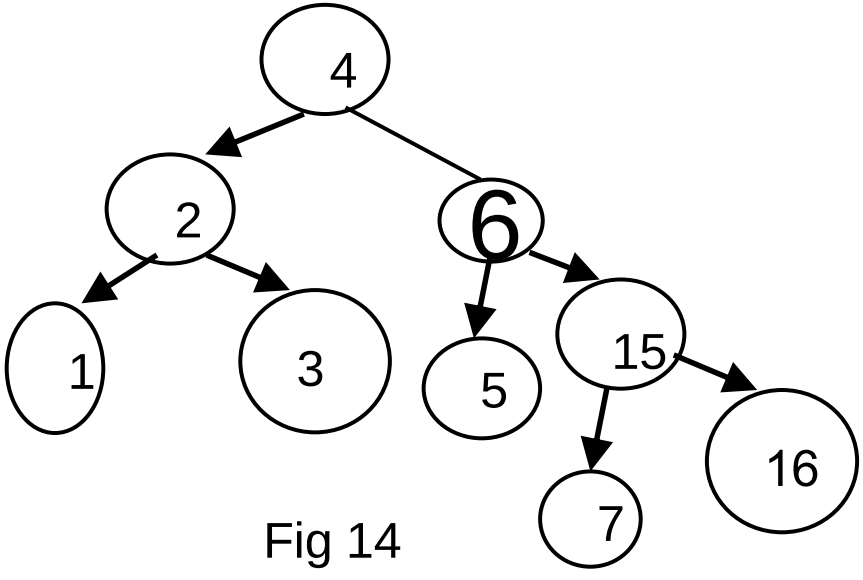


Fig 14

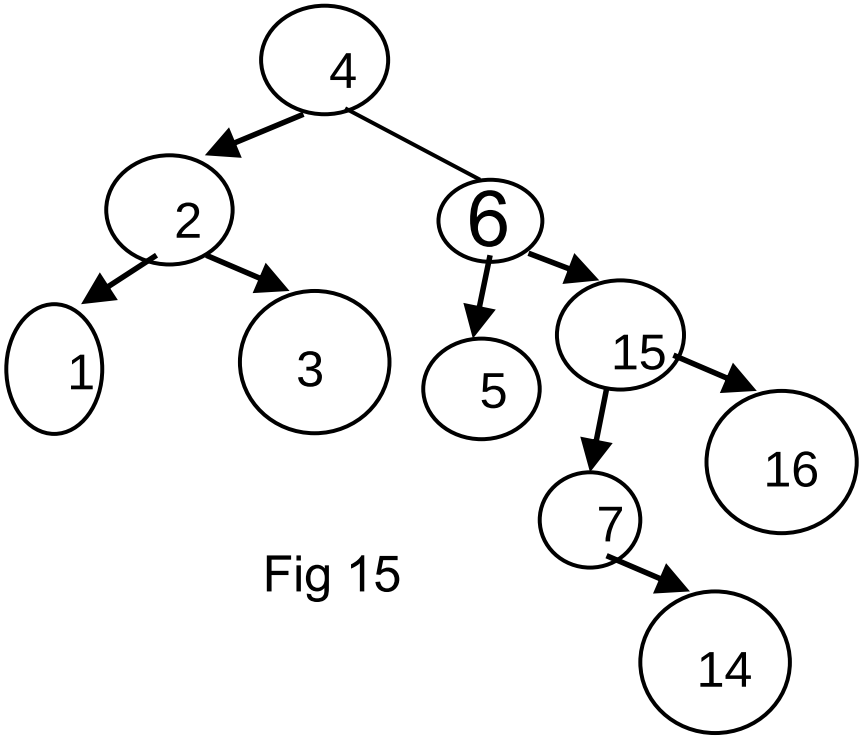


Fig 15

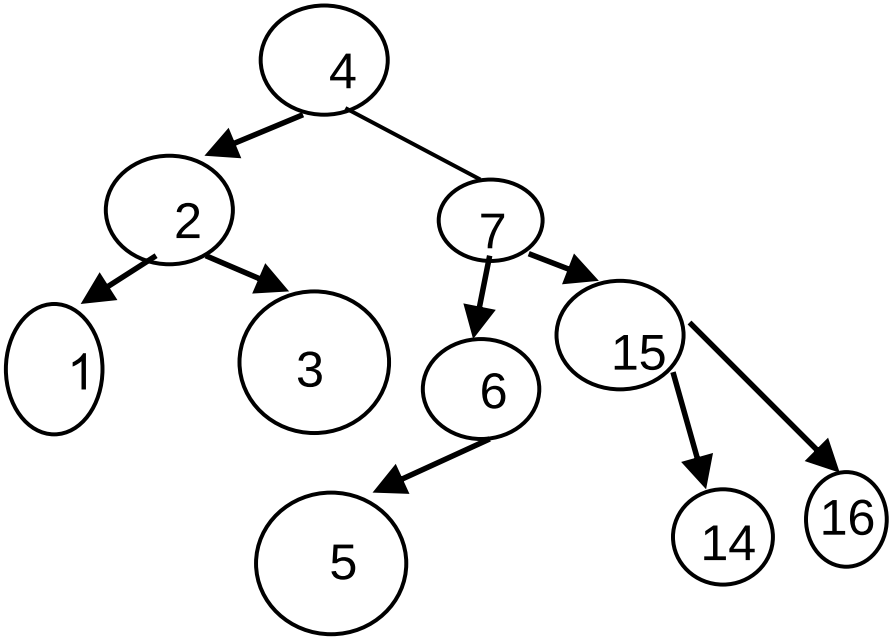


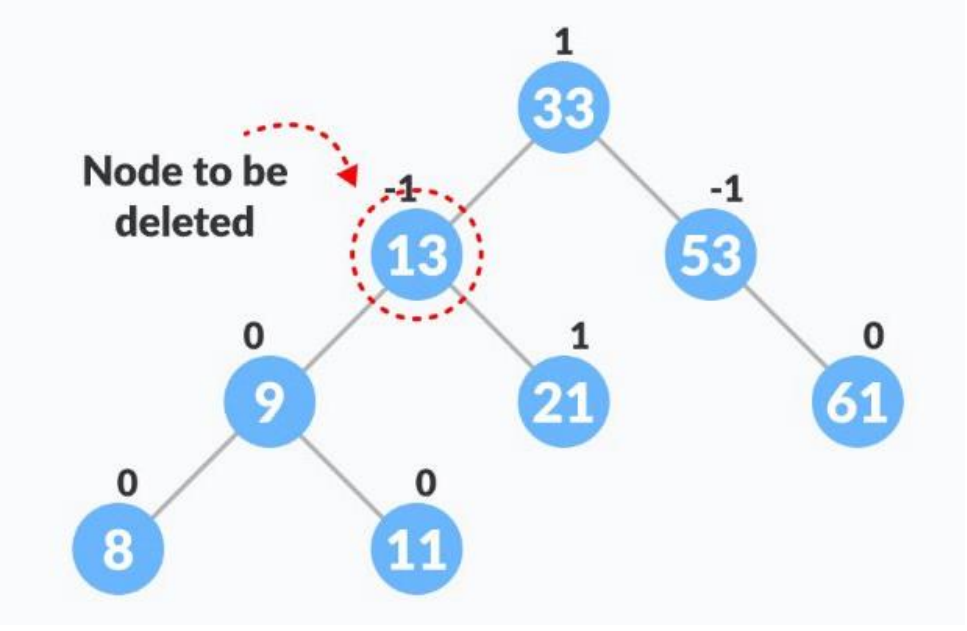
Fig 16

Deletions can be done with similar rotations

Delete on AVL Tree

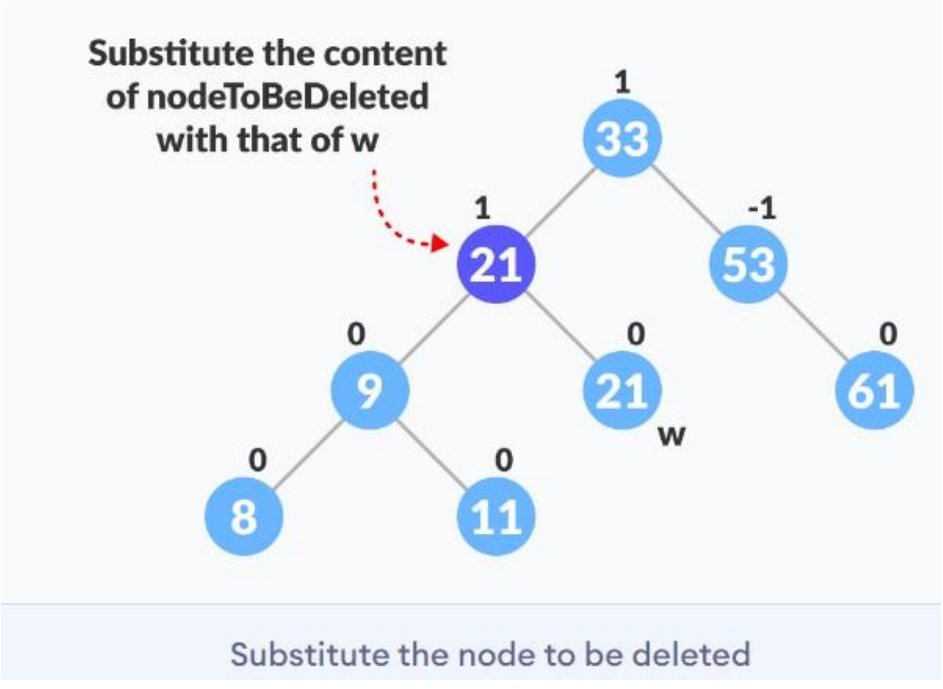
- Delete the node based on the BST's delete procedure
- Starting from the parent of the deleted node, traverse up the tree to the root node, updating the balance factor of each node along the way.
- If a node's balance factor becomes greater than 1 or less than -1 during the traversal, then that node is unbalanced and a rebalancing operation must be performed.
- Depending on the type of imbalance (left-left, left-right, right-right, or right-left), perform a rotation operation to balance the subtree rooted at the unbalanced node.
- After the rotation operation is performed, update the balance factors of the affected nodes.
- Continue the traversal up the tree until the root node is reached, updating the balance factors of each node along the way.
- If the root node's balance factor becomes greater than 1 or less than -1, then perform a rotation operation to balance the entire tree.
- The AVL tree now has the new node inserted, and is still balanced.

Delete

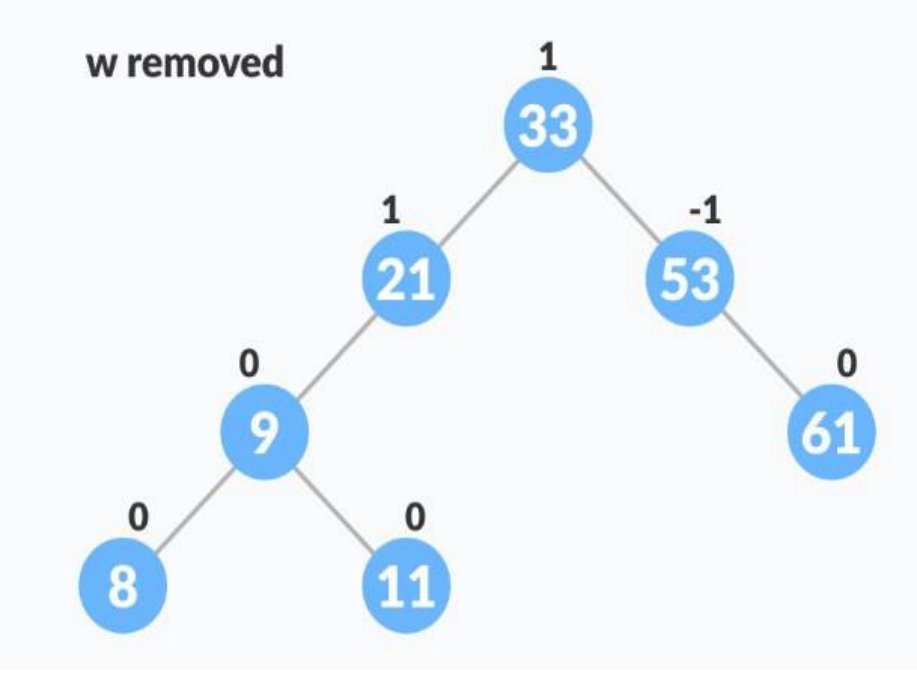


As we have seen in the BST, there are three possible cases for deleting a node:

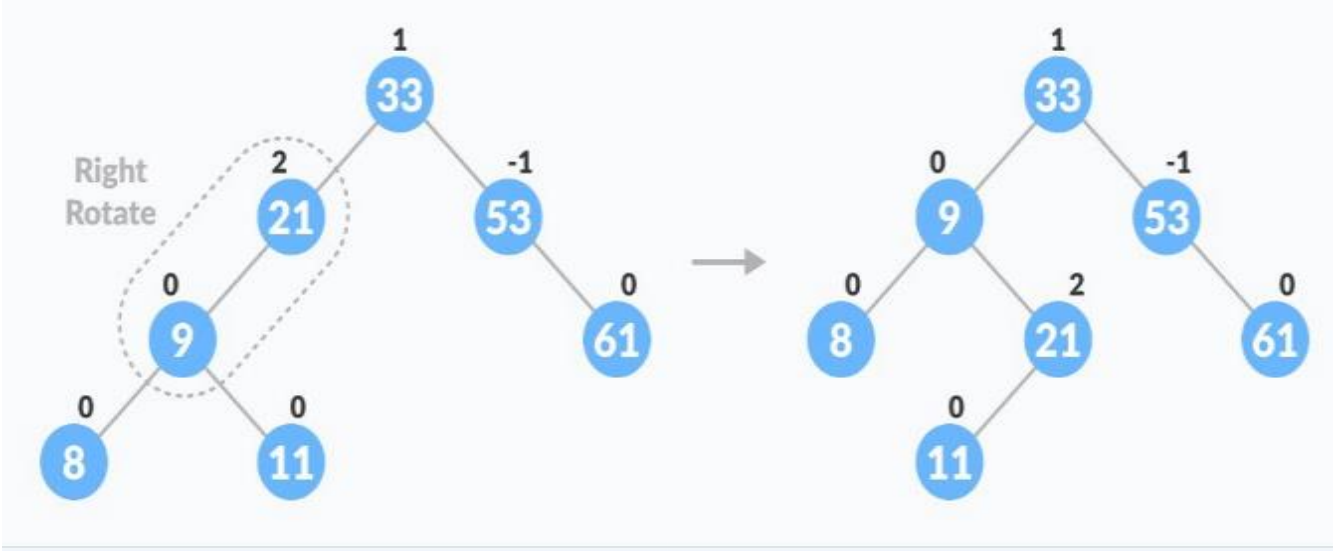
1. Leaf node
2. A node with one child
3. If the node has two children



Substitute the node to be deleted



Delete



Right-rotate for balancing the tree

Question

During the module we've looked at several different ways of implementing a collection of objects, in particular:

- Lists based on arrays
- Linked lists
- Trees, particularly binary search trees
- Hash tables

For each of the following operations, write down the expected time required for the operation. Your answers should assume that the data structure is performing well, i.e., binary search trees are reasonably well balanced and hash tables have a low load factor (i.e., your answer should be the expected time, not the worst case time if those two are different)

***(a) contains*: return true or false depending on whether a particular object is a member of a collection.**

(b) *insert*: add a new item to the collection in the appropriate place

Question

(a) Draw the binary search tree that is created if the following numbers are inserted in the tree in the given order.

12 15 3 35 21 42 14

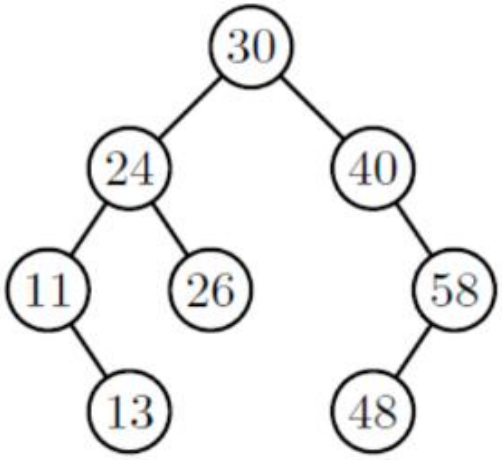
Question

Draw a *balanced* binary search tree containing the same numbers given in part (a).

Question

Consider the Tree below. For each of the algorithms Preorder Tree Walk, In order Tree Walk, and Post order Tree Walk answer the following questions:

(a) Does the algorithm print the keys of elements stored in a binary search tree in a sorted order?

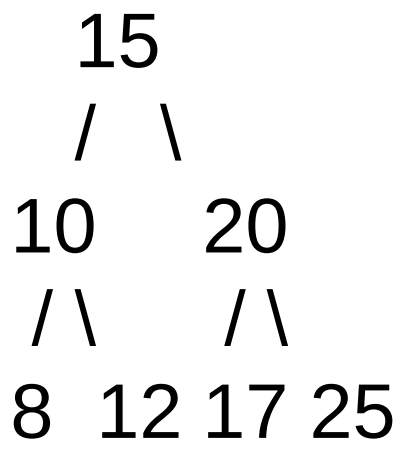


Question

The function **Find(k, R)** shown below is a recursive implementation for searching a key k in a Binary Search Tree (BST) with root node R. Given the following BST:

```
Find(k, R)

if R.Key = k:
  return R
else if R.Key > k :
  return Find(k, R.Left)
else if R.Key < k :
  return Find(k, R.Right)
```



1. Trace the execution of **Find(12, R)** step by step, assuming R is the root of the BST shown above. Which nodes are visited, and what is the final output?
2. Trace the execution of **Find(18, R)** for the same tree. Which nodes are visited, and what is the final output?
3. What is the **time complexity** of this algorithm in the **best-case** and **worst-case** scenarios? Provide examples of BSTs that correspond to these cases.
4. Modify the **Find(k, R)** function to handle the case where R is null (i.e., the tree is empty or the key is not found). Provide the updated pseudocode.

Question

Question: Suppose you have an initially empty AVL tree. Insert the following keys in the given order and show the resulting AVL tree after each insertion. Clearly indicate any rotations (if necessary) to maintain the AVL property.

Keys to insert: 10, 20, 30, 15, 25, 5, 35

Instructions:

1. Perform the insertions one at a time.
2. After each insertion, calculate the balance factor for each node.
3. If the tree becomes unbalanced, identify the type of rotation required (LL, RR, LR, or RL) and perform the rotation.
4. Draw the AVL tree after each step, highlighting any rotations performed.